

Model-Based RL Algorithms

CS 185/285

Instructor: Sergey Levine
UC Berkeley



Part 1:

Planning with models

Last time...

1. Get some data
collected

for now, let's talk about
**planning: using the model to
make decisions without a policy**

2. Learn “simulator”

$$\min_f \sum_i ||f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i||^2$$

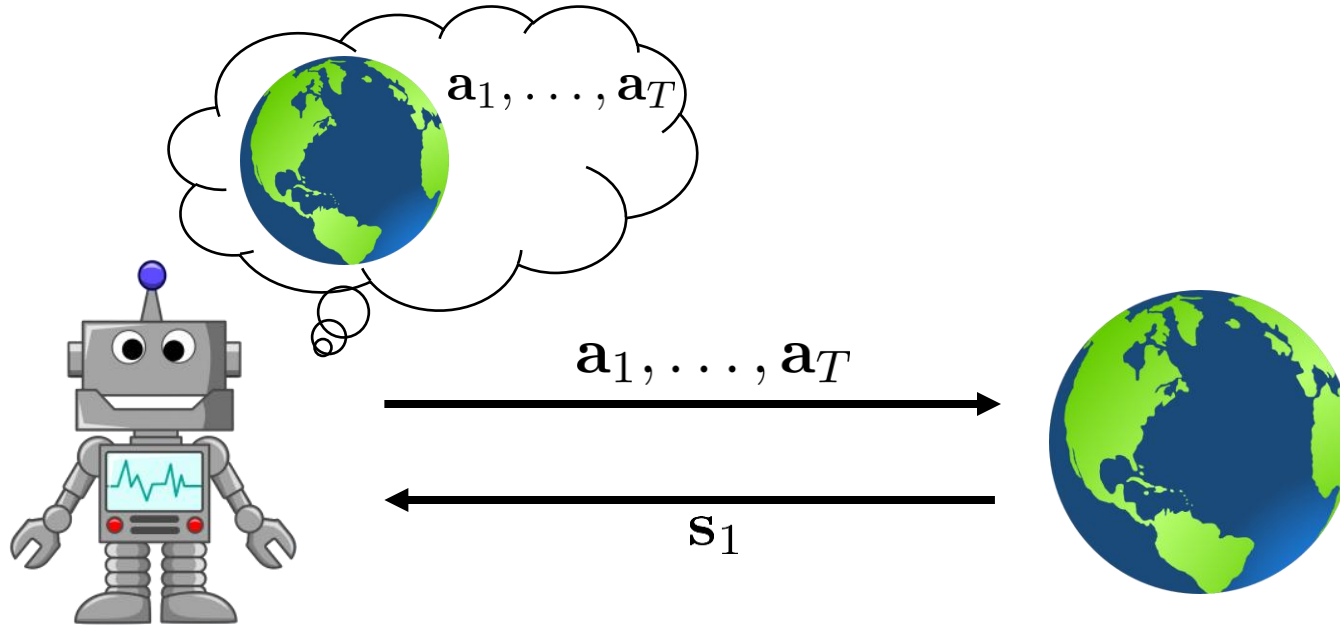
distributional shift

deal with

3. Run your favorite RL method inside “simulator” f

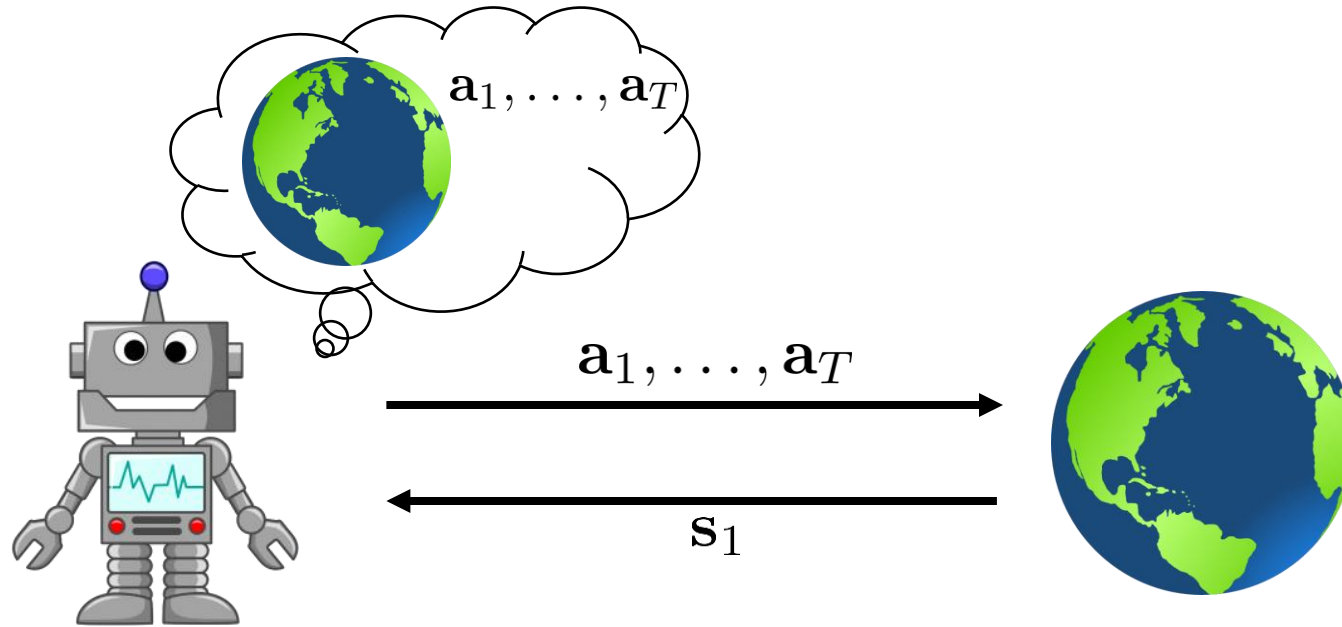
could run one of the algorithms we learned
about, but there are better options here

The deterministic case



$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} \sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \text{ s.t. } \mathbf{a}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$$

The stochastic open-loop case



$$p_{\theta}(\mathbf{s}_1, \dots, \mathbf{s}_T | \mathbf{a}_1, \dots, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^T p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

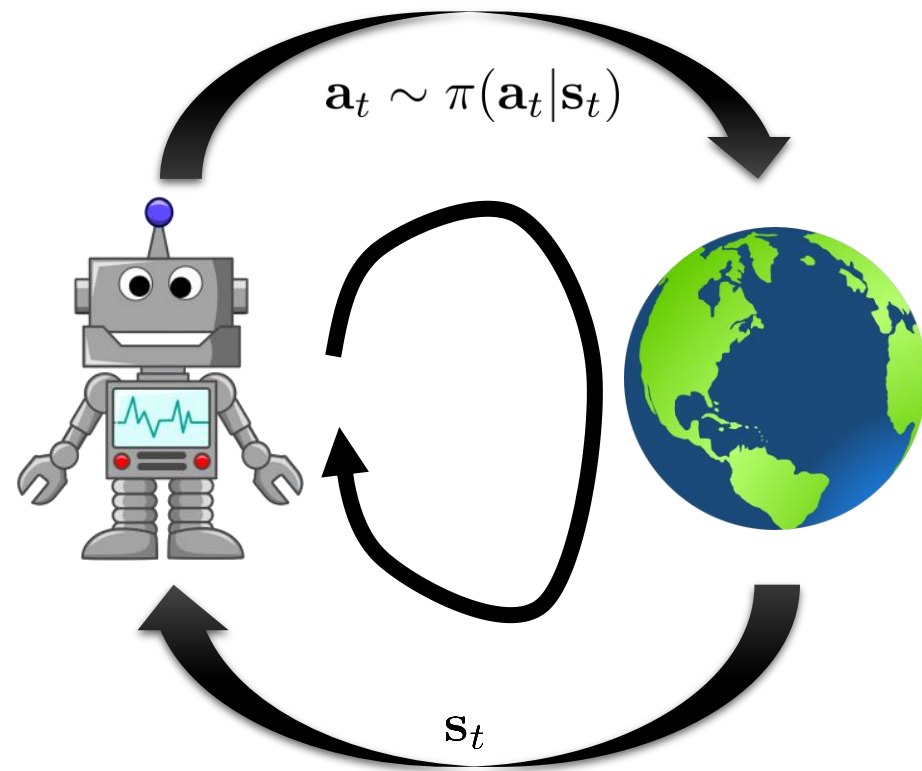
$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} E \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) | \mathbf{a}_1, \dots, \mathbf{a}_T \right]$$

why is this suboptimal?

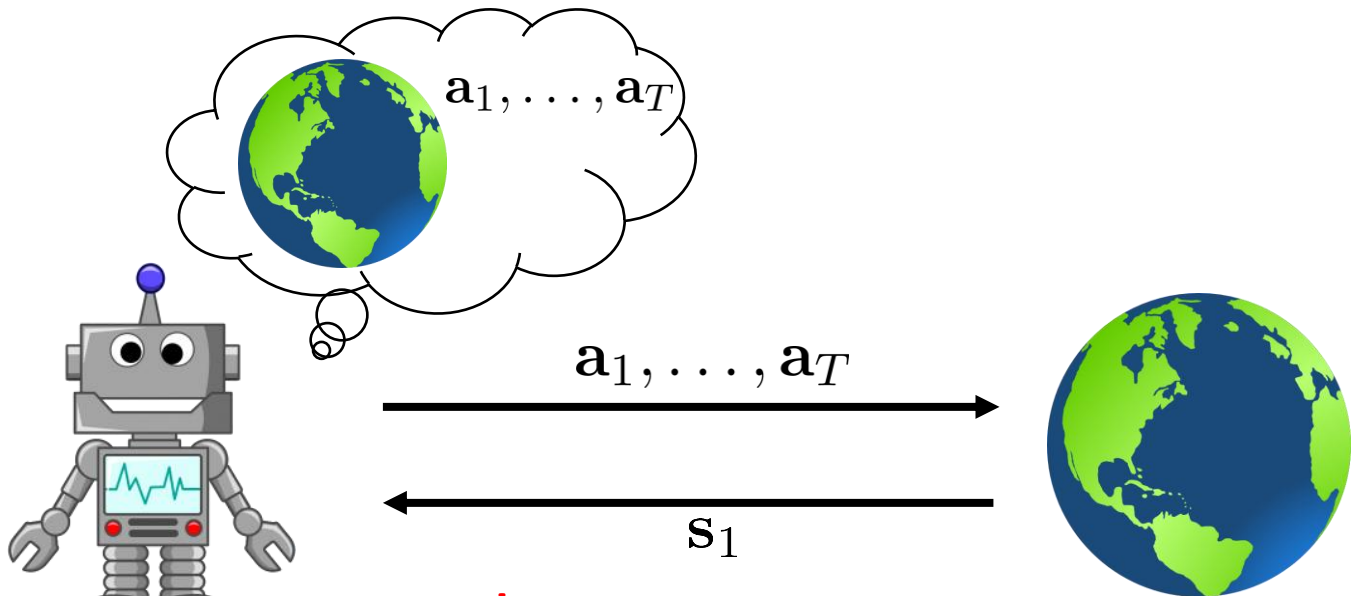
Aside: terminology

what is this “loop”?

closed-loop

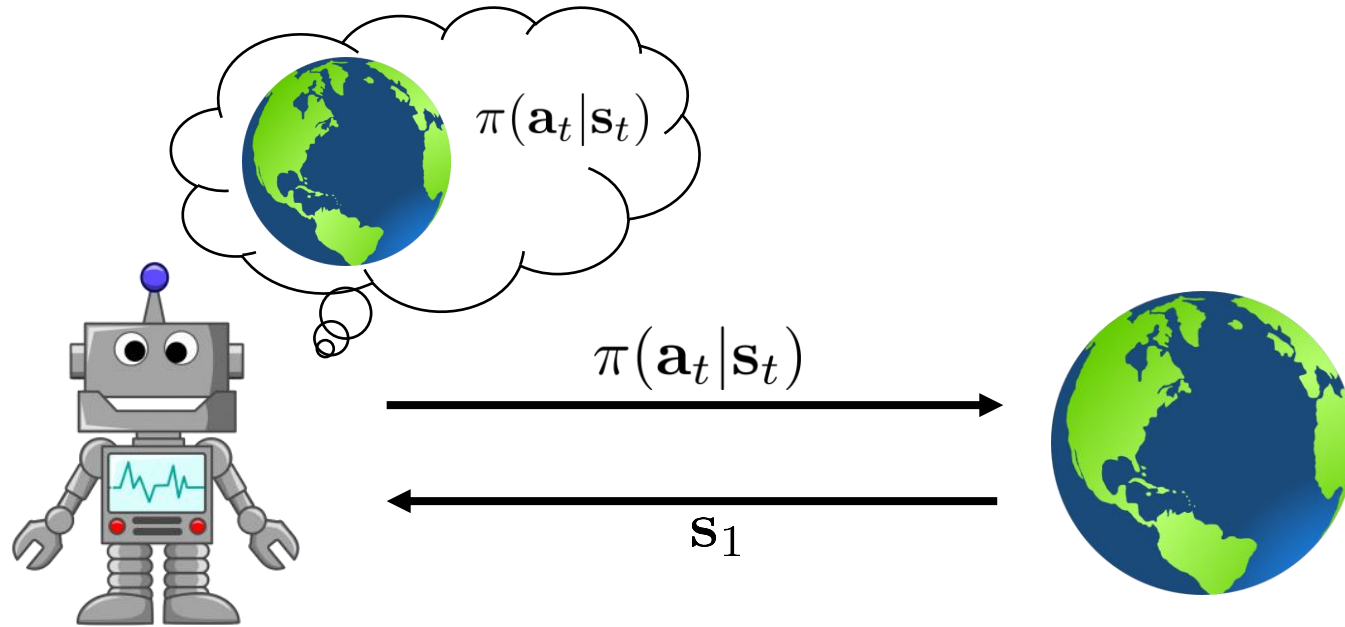


open-loop



only sent at $t = 1$,
then it's one-way!

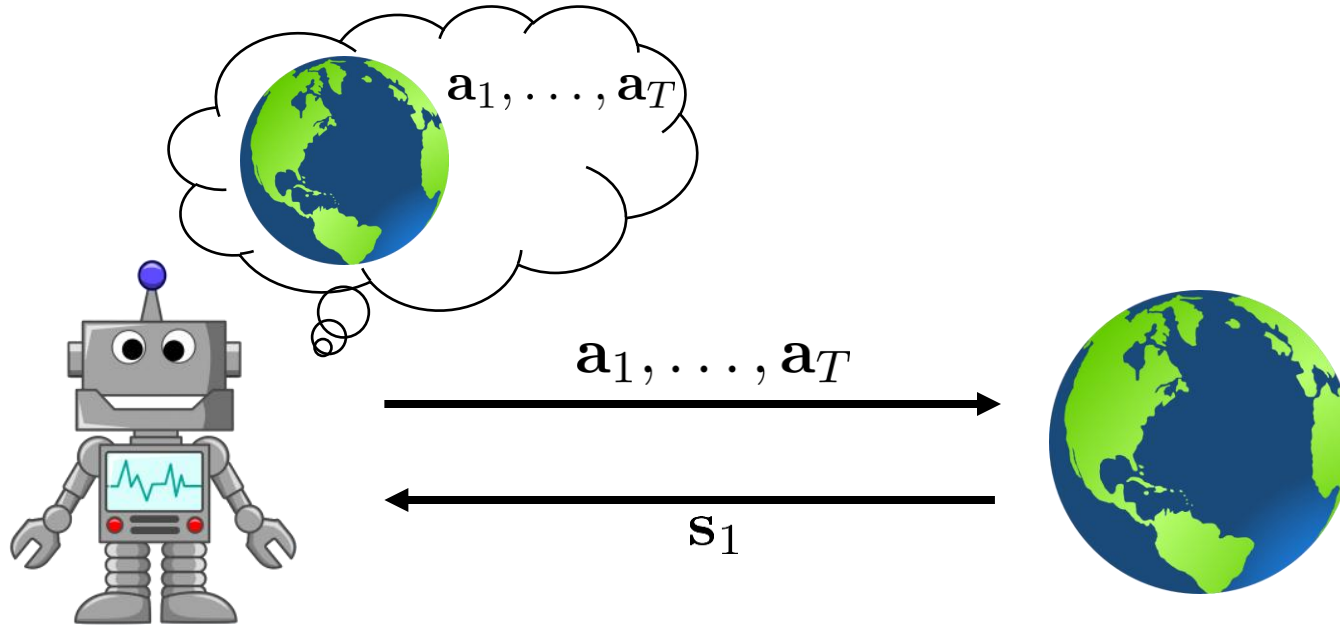
The stochastic **closed-loop** case



$$p(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^T \pi(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\pi = \arg \max_{\pi} E_{\tau \sim p(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

But for now, open-loop planning



$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} \sum_{t=1}^T r(\mathbf{s}_t, \mathbf{a}_t) \text{ s.t. } \mathbf{a}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$$

Stochastic optimization

abstract away optimal control/planning:

$$\mathbf{a}_1, \dots, \mathbf{a}_T = \arg \max_{\mathbf{a}_1, \dots, \mathbf{a}_T} \underbrace{J(\mathbf{a}_1, \dots, \mathbf{a}_T)}$$

don't care what this is

$$\mathbf{A} = \arg \max_{\mathbf{A}} J(\mathbf{A})$$

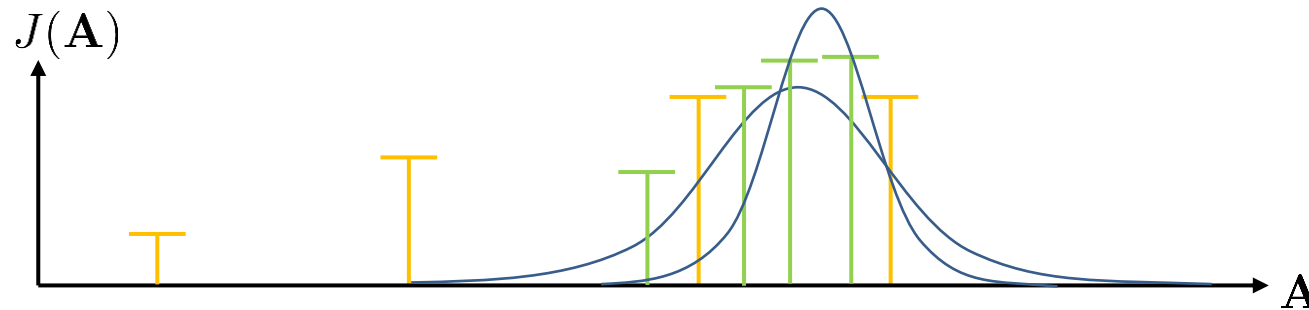
simplest method: guess & check “random shooting method”

1. pick $\mathbf{A}_1, \dots, \mathbf{A}_N$ from some distribution (e.g., uniform)
2. choose $\mathbf{A}_i$ based on $\arg \max_i J(\mathbf{A}_i)$

Cross-entropy method (CEM)

1. pick $\mathbf{A}_1, \dots, \mathbf{A}_N$ from some distribution (e.g., uniform)

2. choose $\mathbf{A}_i$ based on $\arg \max_i J(\mathbf{A}_i)$ can we do better?



typically use Gaussian distribution

see also: CMA-ES (sort of like CEM with momentum)

cross-entropy method with continuous-valued inputs:

1. sample $\mathbf{A}_1, \dots, \mathbf{A}_N$ from $p(\mathbf{A})$
2. evaluate $J(\mathbf{A}_1), \dots, J(\mathbf{A}_N)$
3. pick the *elites* $\mathbf{A}_{i_1}, \dots, \mathbf{A}_{i_M}$ with the highest value, where $M < N$
4. refit $p(\mathbf{A})$ to the elites $\mathbf{A}_{i_1}, \dots, \mathbf{A}_{i_M}$

What's the upside?

1. Very fast if parallelized
2. Extremely simple

Other methods for planning:

Monte Carlo tree search (MCTS)
continuous trajectory optimization (LQR, etc.)
tree-based motion planning (e.g., RRT)

What's the problem?

1. Very harsh dimensionality limit
2. Only open-loop planning

How to plan with uncertainty

Before: $J(\mathbf{a}_1, \dots, \mathbf{a}_H) = \sum_{t=1}^H r(\mathbf{s}_t, \mathbf{a}_t)$, where $\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t)$

Now: $J(\mathbf{a}_1, \dots, \mathbf{a}_H) = \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H r(\mathbf{s}_{t,i}, \mathbf{a}_t)$, where $\mathbf{s}_{t+1,i} = f_i(\mathbf{s}_{t,i}, \mathbf{a}_t)$

↖
distribution over
deterministic models

In general, for candidate action sequence $\mathbf{a}_1, \dots, \mathbf{a}_H$:

Step 1: sample $\theta \sim p(\theta|\mathcal{D})$

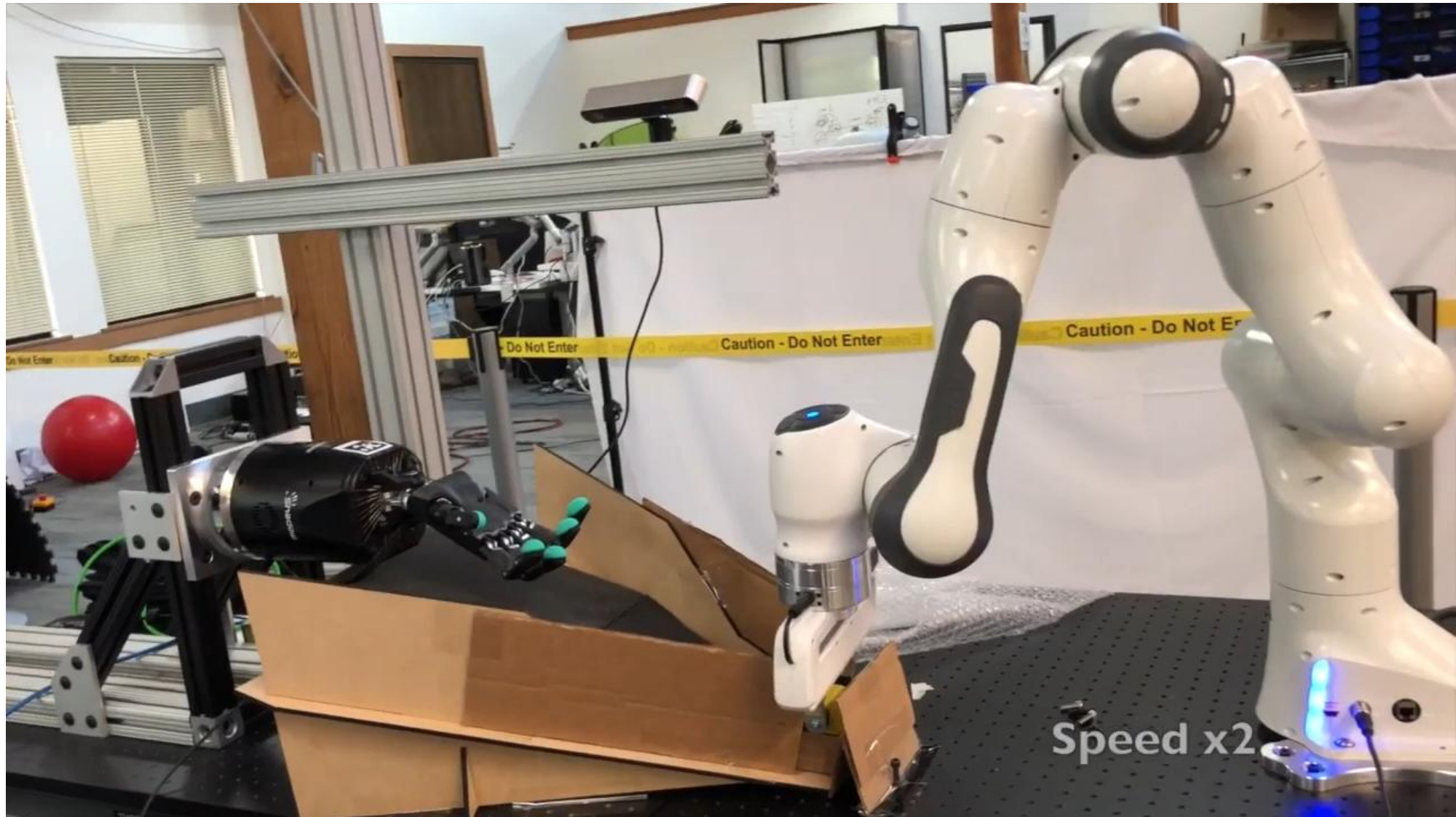
Step 2: at each time step t , sample $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, \theta)$

Step 3: calculate $R = \sum_t r(\mathbf{s}_t, \mathbf{a}_t)$

Step 4: repeat steps 1 to 3 and accumulate the average reward

Other options: moment matching, more complex posterior estimation with BNNs, etc.

Model-based RL in the real world example



Part 2:

Policy learning with models

Notional “learned simulator” algorithm

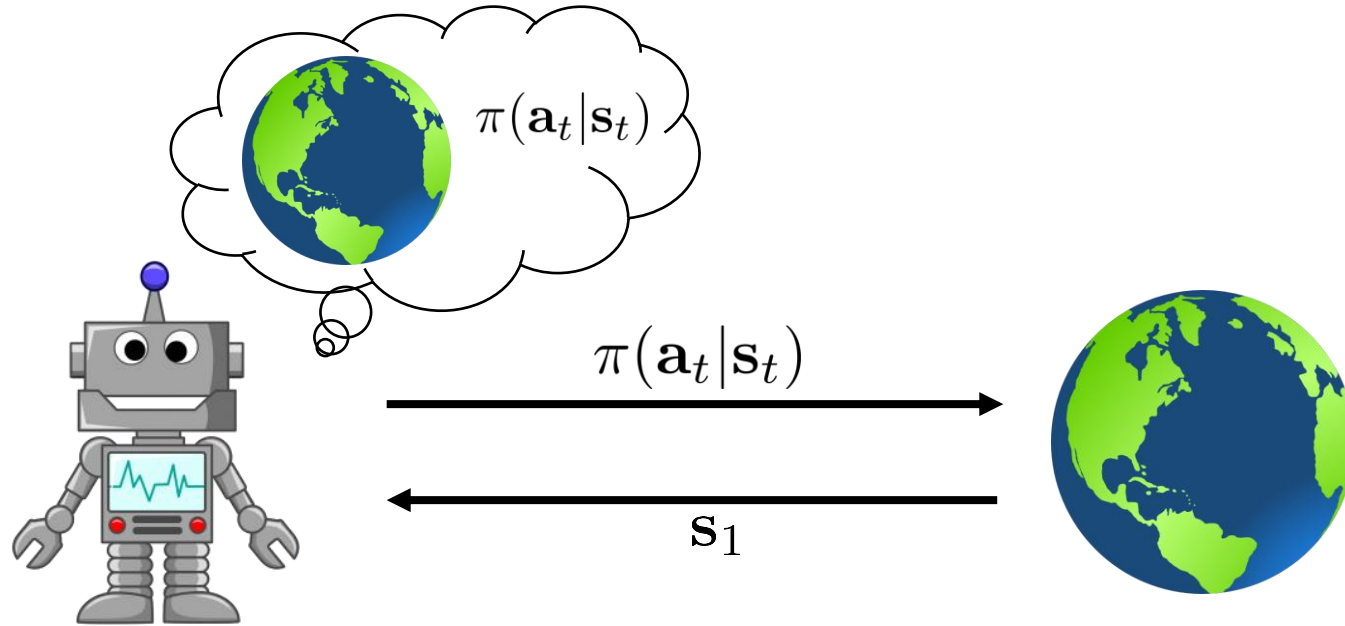
1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$
collected using base policy $\pi_\beta(\mathbf{a}|\mathbf{s})$

2. Learn “simulator” $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$

$$\min_f \sum_i ||f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i||^2$$

3. Run your favorite RL method inside “simulator” f

The stochastic **closed**-loop case



$$p(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T) = p(\mathbf{s}_1) \prod_{t=1}^T \pi(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$$

$$\pi = \arg \max_{\pi} E_{\tau \sim p(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Model-free optimization with a model

Policy gradient:
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_{i,t} | \mathbf{s}_{i,t}) \hat{Q}_{i,t}^{\pi}$$

Backprop (pathwise) gradient:
$$\nabla_{\theta} J(\theta) = \sum_{t=1}^H \frac{d\mathbf{a}_t}{d\theta} \frac{d\mathbf{s}_{t+1}}{d\mathbf{a}_t} \left(\sum_{t'=t+1}^H \frac{dr_{t'}}{d\mathbf{s}_{t'}} \left(\prod_{t''=t+2}^{t'} \frac{d\mathbf{s}_{t''}}{d\mathbf{a}_{t''-1}} \frac{d\mathbf{a}_{t''-1}}{d\mathbf{s}_{t''-1}} + \frac{d\mathbf{s}_{t''}}{d\mathbf{s}_{t''-1}} \right) \right)$$

- Policy gradient might be more *stable* (if enough samples are used) because it does not require multiplying many Jacobians
- See some analysis here:
 - Parmas et al. '18: PIPP: Flexible Model-Based Policy Search Robust to the Curse of Chaos

Model-based RL via policy gradient

model-based reinforcement learning version 2.5:

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$

2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$

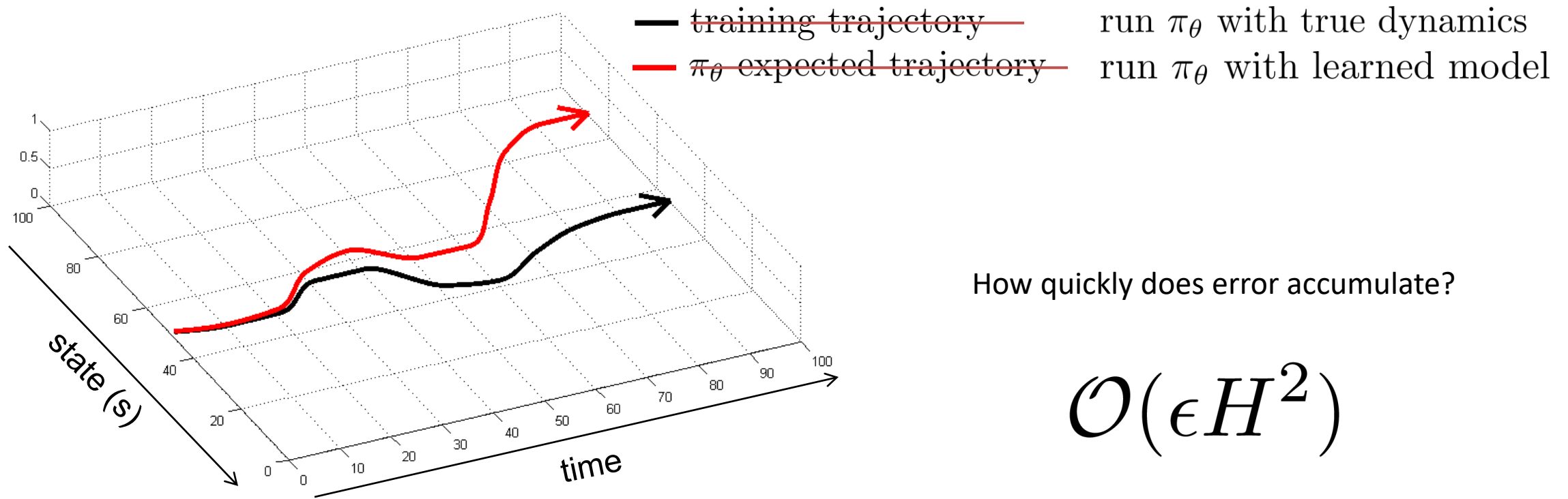
3. use $f(\mathbf{s}, \mathbf{a})$ to generate trajectories $\{\tau_i\}$ with policy $\pi_\theta(\mathbf{a}|\mathbf{s})$

4. use $\{\tau_i\}$ to improve $\pi_\theta(\mathbf{a}|\mathbf{s})$ via policy gradient

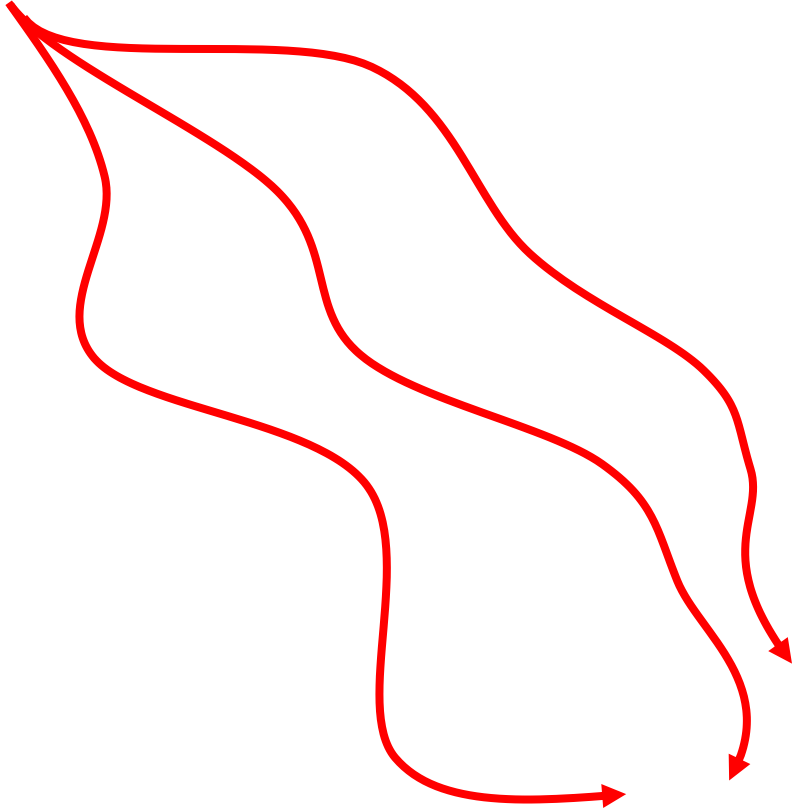
5. run $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$, appending the visited tuples $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to $\mathcal{D}$

What's a potential **problem** with this approach?

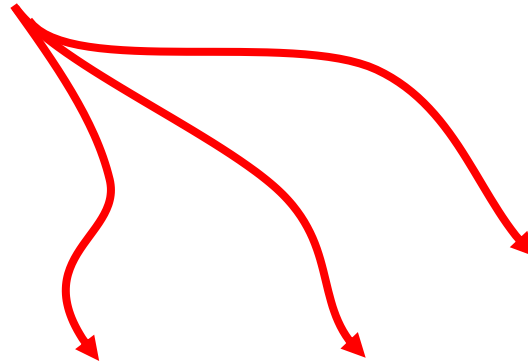
The curse of long model-based rollouts



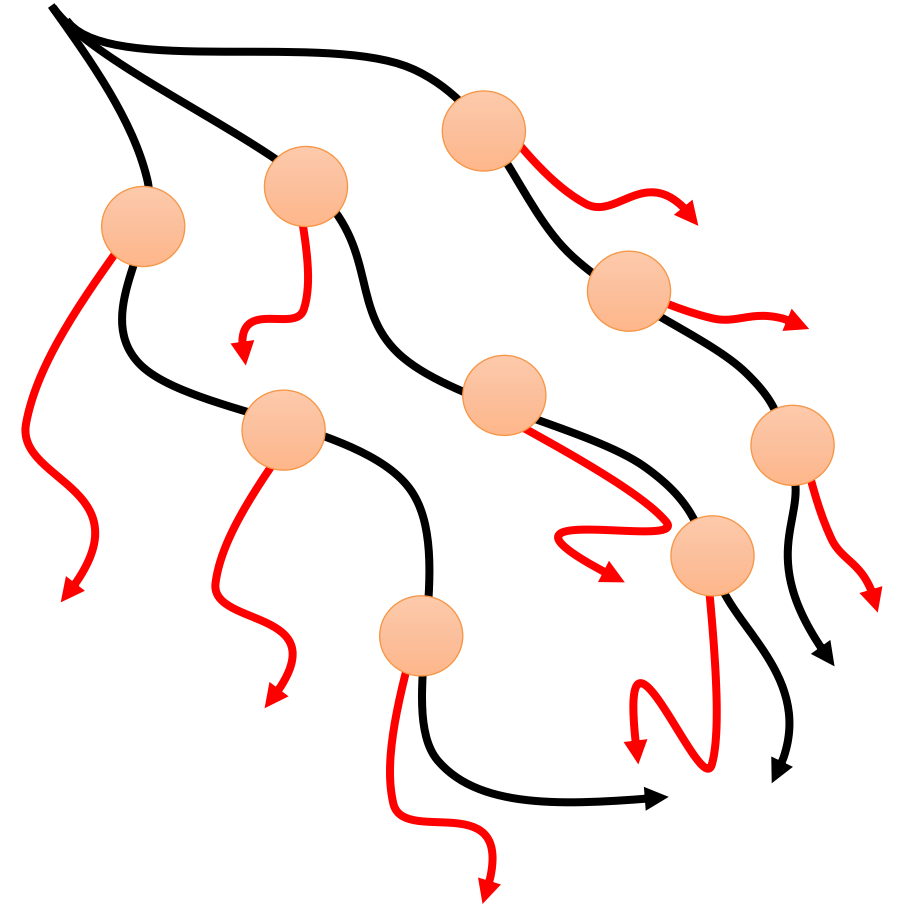
How to get away with **short** rollouts?



- huge accumulating error



+ much lower error
- never see later time steps



+ much lower error
+ see all time steps
- wrong state distribution

Model-based RL with **short** rollouts

model-based reinforcement learning version 3.0:

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{s}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}')_i\}$
2. learn dynamics model $f(\mathbf{s}, \mathbf{a})$ to minimize $\sum_i \|f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i\|^2$
3. pick states $\mathbf{s}_i$ from $\mathcal{D}$, use $f(\mathbf{s}, \mathbf{a})$ to make *short* rollouts from them
4. use *both* real and model data to improve $\pi_\theta(\mathbf{a}|\mathbf{s})$ with *off-policy RL*
5. run $\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)$, appending the visited tuples $(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ to $\mathcal{D}$

Model-free optimization with a model

Dyna

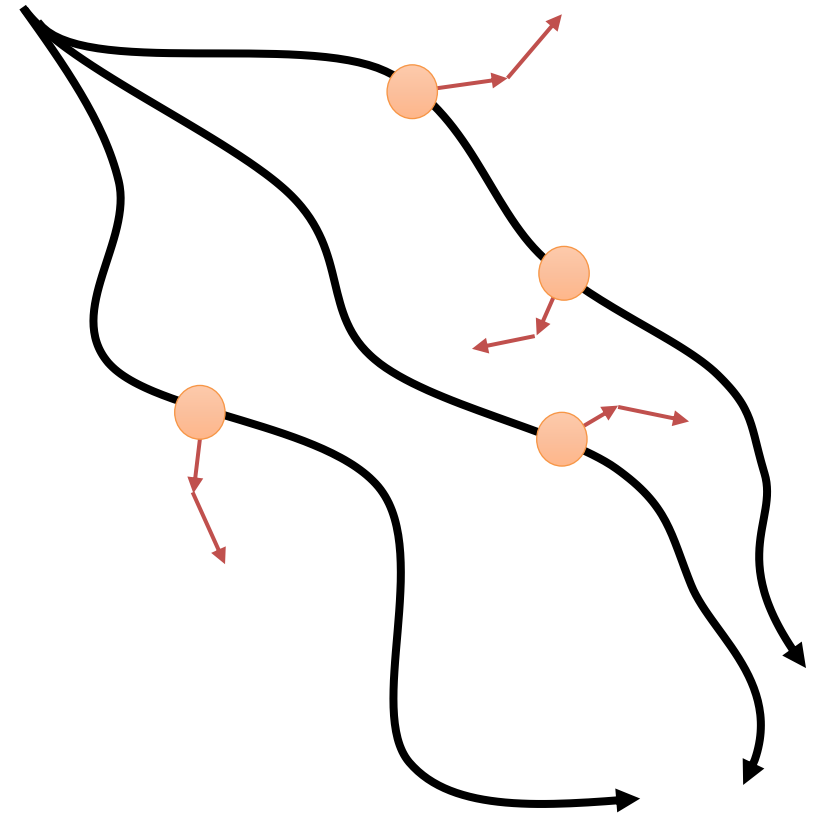
online Q-learning algorithm that performs model-free RL with a model

1. given state s , pick action a using exploration policy
2. observe s' and r , to get transition (s, a, s', r)
3. update model $\hat{p}(s'|s, a)$ and $\hat{r}(s, a)$ using (s, a, s')
4. Q-update: $Q(s, a) \leftarrow Q(s, a) + \alpha E_{s', r}[r + \max_{a'} Q(s', a') - Q(s, a)]$
5. repeat K times:
 6. sample $(s, a) \sim \mathcal{B}$ from buffer of past states and actions
 7. Q-update: $Q(s, a) \leftarrow Q(s, a) + \alpha E_{s', r}[r + \max_{a'} Q(s', a') - Q(s, a)]$

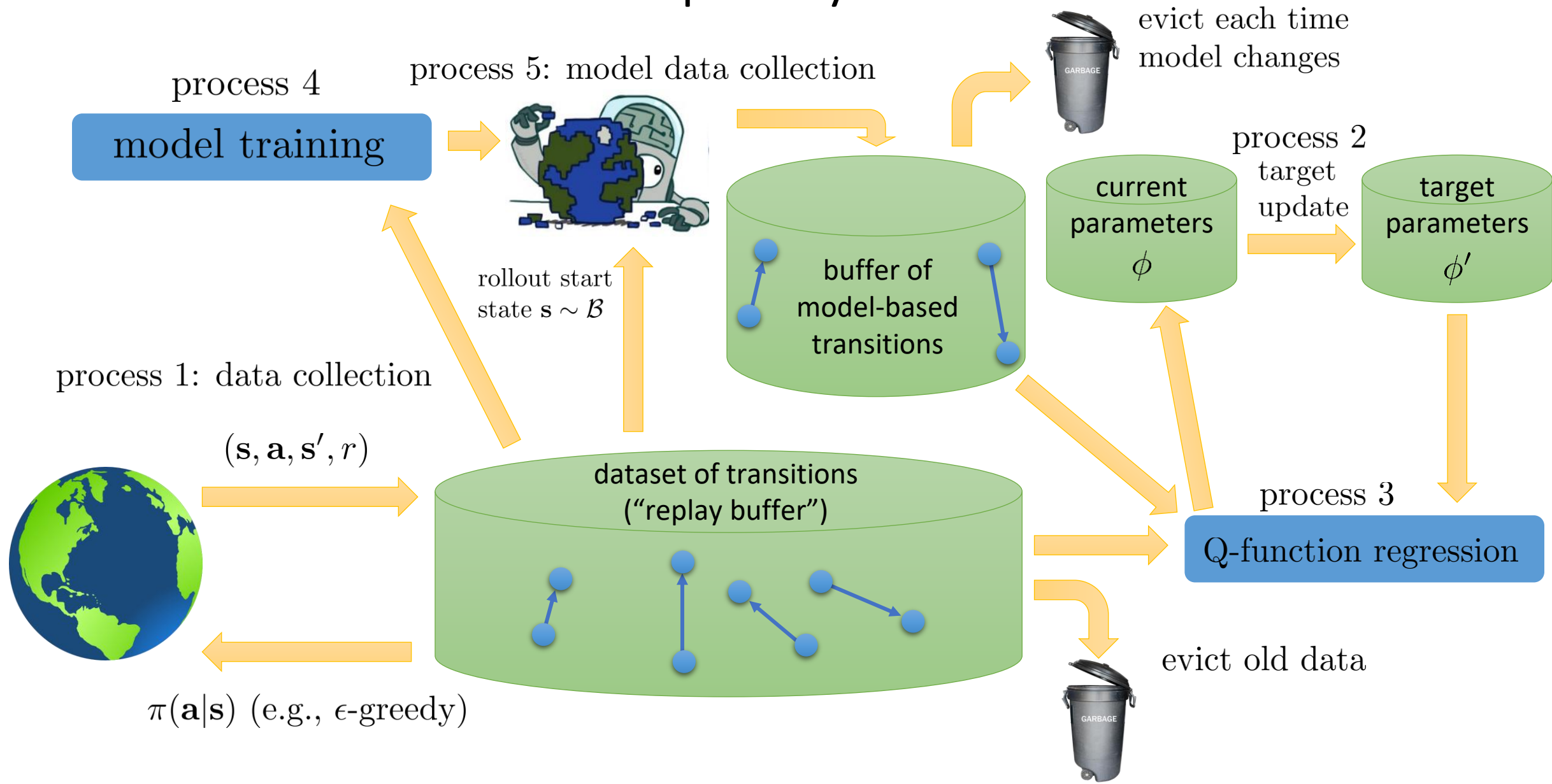
General “Dyna-style” model-based RL recipe

1. collect some data, consisting of transitions (s, a, s', r)
2. learn model $\hat{p}(s'|s, a)$ (and optionally, $\hat{r}(s, a)$)
3. repeat K times:
 4. sample $s \sim \mathcal{B}$ from buffer
 5. choose action a (from $\mathcal{B}$, from π , or random)
 6. simulate $s' \sim \hat{p}(s'|s, a)$ (and $r = \hat{r}(s, a)$)
 7. train on (s, a, s', r) with model-free RL
 8. (optional) take N more model-based steps

+ only requires short (as few as one step) rollouts from model
+ still sees diverse states



Model-accelerated off-policy RL



Model-Based Acceleration (MBA)

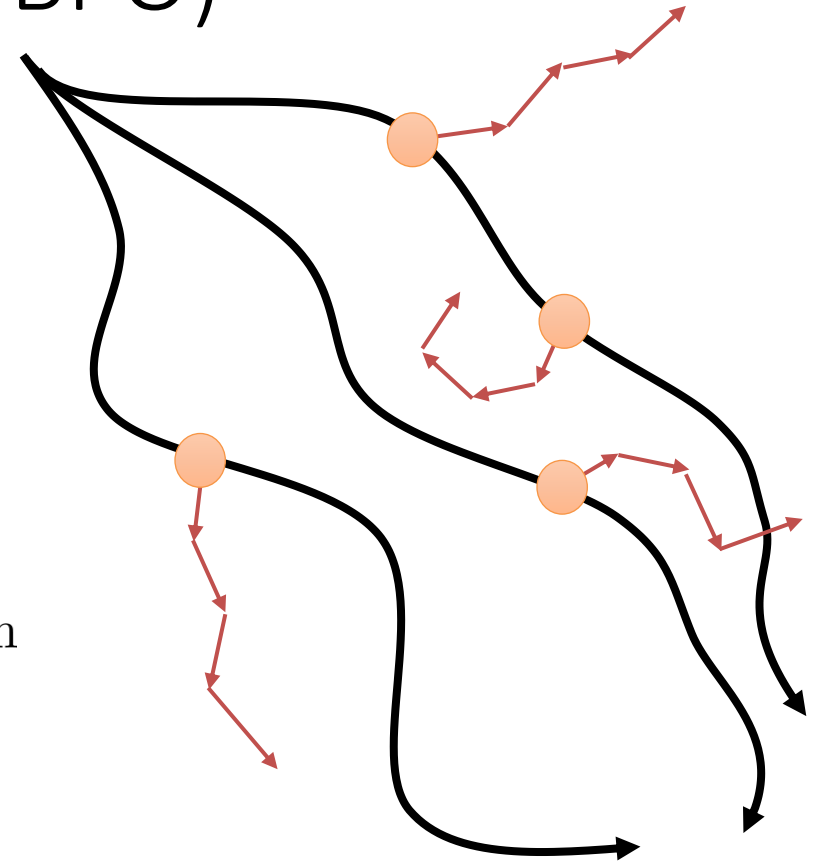
Model-Based Value Expansion (MVE)

Model-Based Policy Optimization (MBPO)

1. take some action $\mathbf{a}_i$ and observe $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, r_i)$, add it to $\mathcal{B}$
2. sample mini-batch $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j, r_j\}$ from $\mathcal{B}$ uniformly
3. use $\{\mathbf{s}_j, \mathbf{a}_j, \mathbf{s}'_j\}$ to update model $\hat{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$
4. sample $\{\mathbf{s}_j\}$ from $\mathcal{B}$
5. for each $\mathbf{s}_j$, perform model-based rollout with $\mathbf{a} = \pi(\mathbf{s})$
6. use all transitions $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$ along rollout to update Q-function

+ why is this a *good* idea?

- why is this a *bad* idea?



Gu et al. Continuous deep Q-learning with model-based acceleration. '16

Feinberg et al. Model-based value expansion. '18

Janner et al. When to trust your model: model-based policy optimization. '19

Intermission

Part 3:

Representing the model

Notional “learned simulator” algorithm

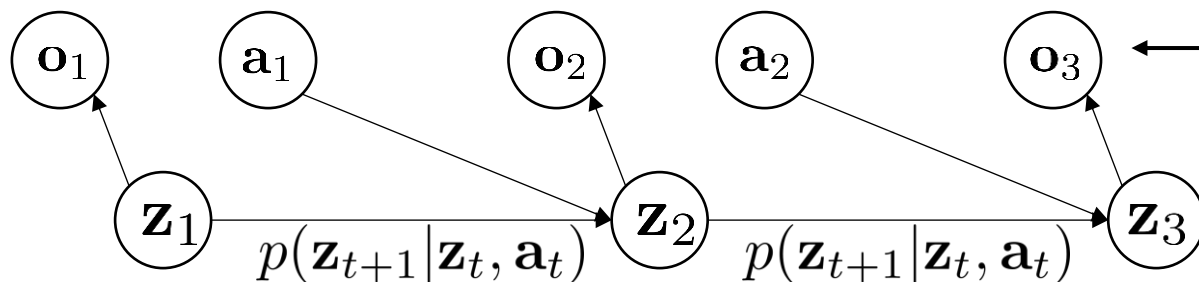
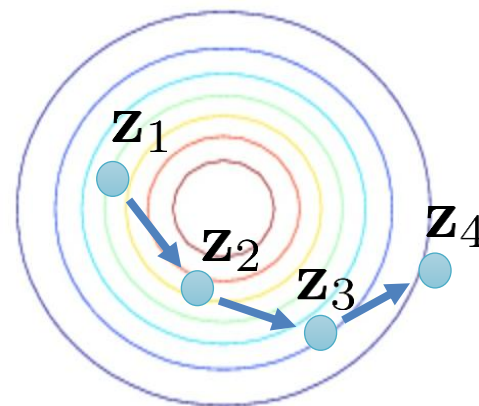
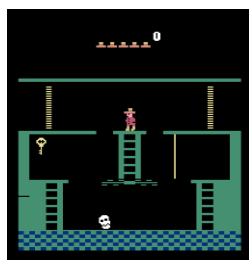
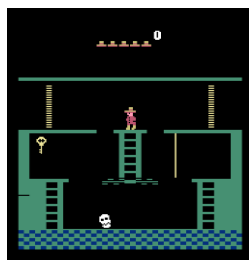
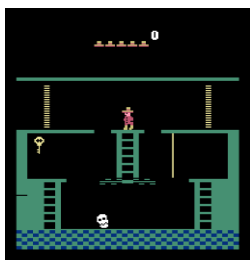
1. Get some data $\mathcal{D} = \{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i\}$
collected using base policy $\pi_\beta(\mathbf{a}|\mathbf{s})$

2. Learn “simulator” $f(\mathbf{s}, \mathbf{a}) = \mathbf{s}'$

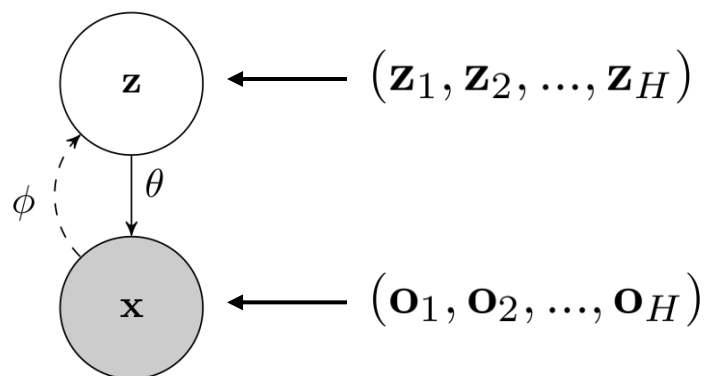
$$\min_f \sum_i ||f(\mathbf{s}_i, \mathbf{a}_i) - \mathbf{s}'_i||^2$$

3. Run your favorite RL method inside “simulator” f

Latent state space models



we are not in partially observed setting



What is the **prior**?

What is the **decoder**?

What is the **encoder**?

Diagram illustrating the prior distribution $\mathcal{N}(0, \mathbf{I})$ and the learned joint distribution:

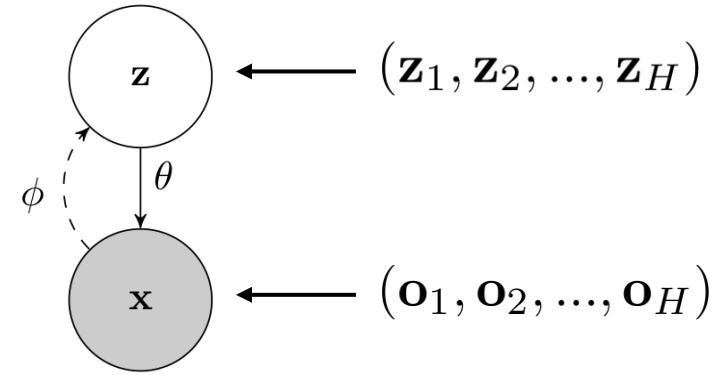
$$p(\mathbf{z}) = p(\mathbf{z}_1) \prod_t p(\mathbf{z}_{t+1} | \mathbf{z}_t, \mathbf{a}_t)$$

The prior $\mathcal{N}(0, \mathbf{I})$ is associated with $p(\mathbf{z}_1)$, and the term $p(\mathbf{z}_{t+1} | \mathbf{z}_t, \mathbf{a}_t)$ is labeled as "learned".

$$p_{\theta}(\mathbf{o} | \mathbf{z}) = \prod_t p(\mathbf{o}_t | \mathbf{z}_t)$$

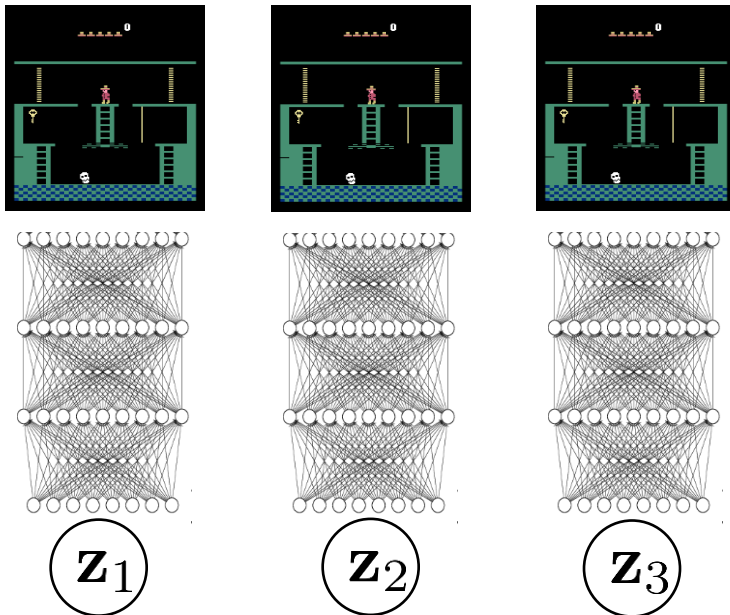
$$q_{\phi}(\mathbf{z} | \mathbf{o}) = \prod_t q_{\phi}(\mathbf{z}_t | \mathbf{o}_{1:t})$$

Latent state space models



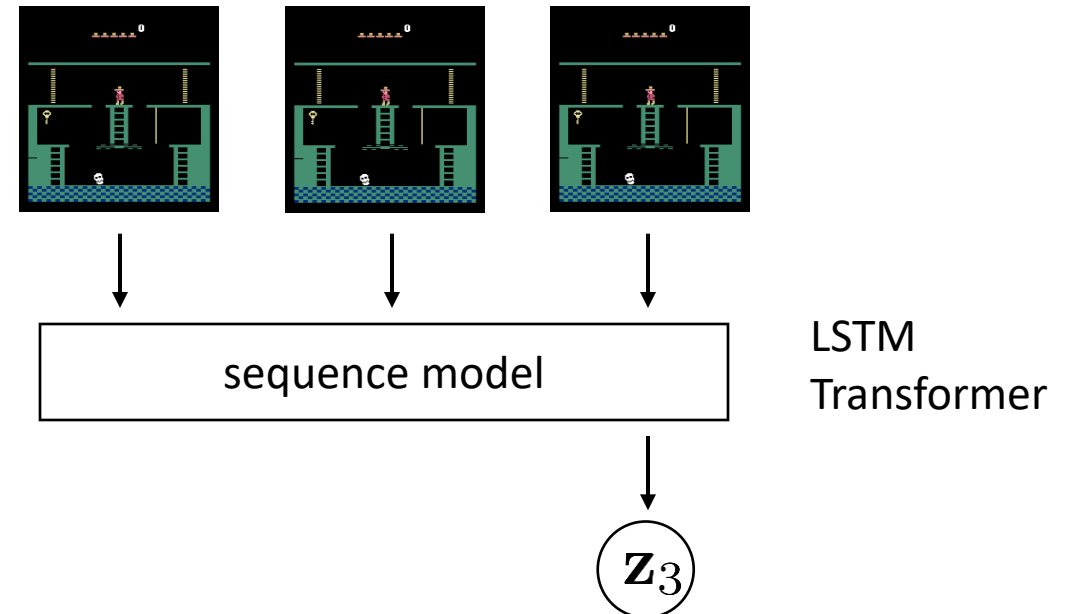
What is the **decoder**?

$$p_{\theta}(\mathbf{o}|\mathbf{z}) = \prod_t p(\mathbf{o}_t|\mathbf{z}_t)$$

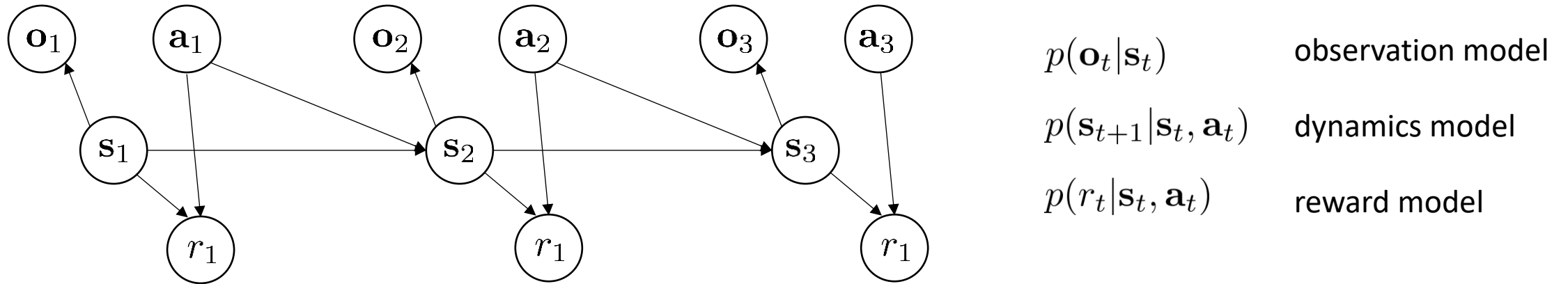


What is the **encoder**?

$$q_{\phi}(\mathbf{z}|\mathbf{o}) = \prod_t q_{\phi}(\mathbf{z}_t|\mathbf{o}_{1:t})$$



State space (latent space) models



How to train?

standard (fully observed) model: $\max_{\phi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \log p_{\phi}(s_{t+1,i} | s_{t,i}, a_{t,i})$

latent space model: $\max_{\phi, \psi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H E_q [\log p_{\phi}(s_{t+1,i} | s_{t,i}, a_{t,i}) + \log p_{\phi}(o_{t,i} | s_{t,i})] + \mathcal{H}(q_{\psi}(s_t, s_{t+1} | o_{1:H,i}, a_{1:H,i}))$

expectation w.r.t. $(s_t, s_{t+1}) \sim q_{\psi}(s_t, s_{t+1} | o_{1:H,i}, a_{1:H,i})$

Model-based RL with latent space models

$$\max_{\phi, \psi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H E_q [\log p_{\phi}(\mathbf{s}_{t+1,i} | \mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \log p_{\phi}(\mathbf{o}_{t,i} | \mathbf{s}_{t,i})] + \mathcal{H}(q_{\psi}(\mathbf{s}_t, \mathbf{s}_{t+1} | \mathbf{o}_{1:H,i}, \mathbf{a}_{1:H,i}))$$

expectation w.r.t. $(\mathbf{s}_t, \mathbf{s}_{t+1}) \sim q_{\psi}(\mathbf{s}_t, \mathbf{s}_{t+1} | \mathbf{o}_{1:H,i}, \mathbf{a}_{1:H,i})$

learn approximate posterior $q_{\psi}(\mathbf{s}_t | \mathbf{o}_{1:t}, \mathbf{a}_{1:t})$ “encoder”

many other choices for approximate posterior:

$q_{\psi}(\mathbf{s}_t, \mathbf{s}_{t+1} | \mathbf{o}_{1:H}, \mathbf{a}_{1:H})$ full smoothing posterior + most accurate
- most complicated

$q_{\psi}(\mathbf{s}_t | \mathbf{o}_t)$ single-step encoder + simplest
- least accurate

we'll talk about this one for now

Model-based RL with latent space models

$$\max_{\phi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H E_q [\log p_{\phi}(\mathbf{s}_{t+1,i} | \mathbf{s}_{t,i}, \mathbf{a}_{t,i}) + \log p_{\phi}(\mathbf{o}_{t,i} | \mathbf{s}_{t,i})] + \mathcal{H}(q(\mathbf{s}_t | \mathbf{o}_t))$$

expectation w.r.t. $\mathbf{s}_t \sim q(\mathbf{s}_t | \mathbf{o}_t), \mathbf{s}_{t+1} \sim q(\mathbf{s}_{t+1} | \mathbf{o}_{t+1})$

why might this type of encoder be a **bad** idea?

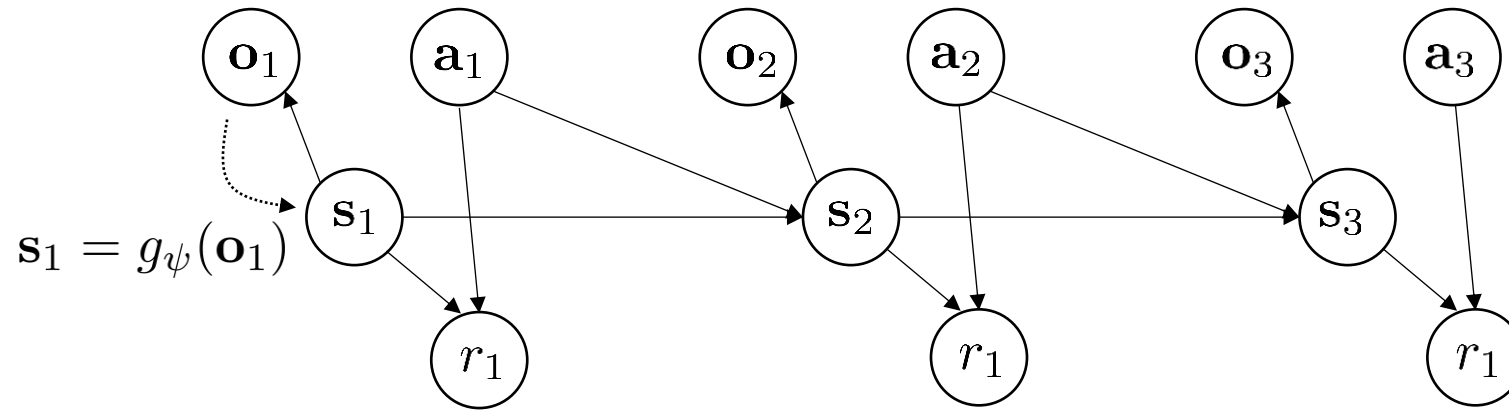
simple special case: $q(\mathbf{s}_t | \mathbf{o}_t)$ is *deterministic*

$q_{\psi}(\mathbf{s}_t | \mathbf{o}_t) = \delta(\mathbf{s}_t = g_{\psi}(\mathbf{o}_t)) \Rightarrow \mathbf{s}_t = g_{\psi}(\mathbf{o}_t)$ deterministic encoder

$$\max_{\phi, \psi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \log p_{\phi}(g_{\psi}(\mathbf{o}_{t+1,i}) | g_{\psi}(\mathbf{o}_{t,i}), \mathbf{a}_{t,i}) + \log p_{\phi}(\mathbf{o}_{t,i} | g_{\psi}(\mathbf{o}_{t,i}))$$

why might *this* type of encoder be a **bad** idea?

Model-based RL with latent space models

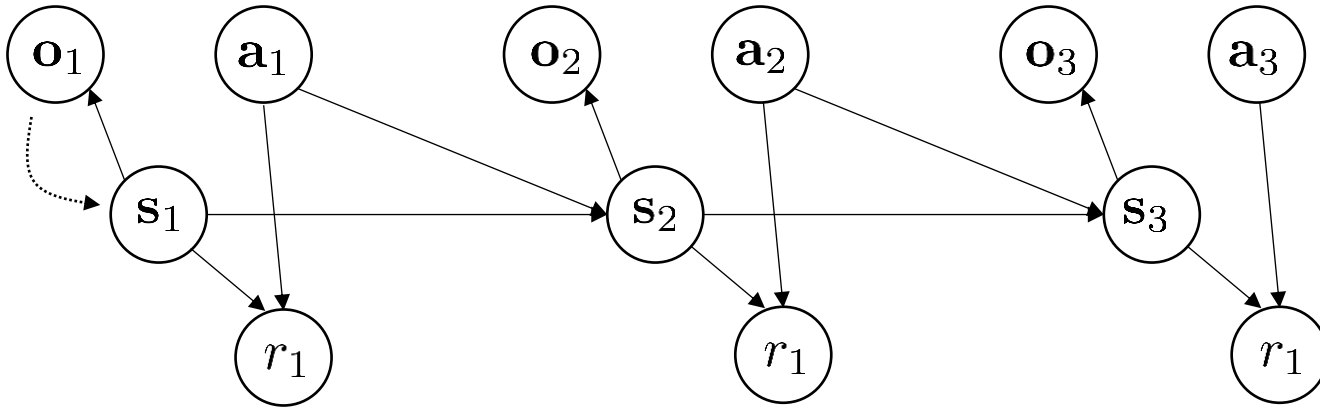


$$\max_{\phi, \psi} \frac{1}{N} \sum_{i=1}^N \sum_{t=1}^H \log p_\phi(g_\psi(\mathbf{o}_{t+1,i}) | g_\psi(\mathbf{o}_{t,i}), \mathbf{a}_{t,i}) + \log p_\phi(\mathbf{o}_{t,i} | g_\psi(\mathbf{o}_{t,i})) + \log p_\phi(r_{t,i} | g_\psi(\mathbf{o}_{t,i}))$$

latent space dynamics image reconstruction reward model

many practical methods use a stochastic encoder, or a more sophisticated smoothing posterior (esp. with partial observability)

Model-based RL with latent space models



model-based reinforcement learning with latent state:

1. run base policy $\pi_0(\mathbf{a}_t|\mathbf{o}_t)$ (e.g., random policy) to collect $\mathcal{D} = \{(\mathbf{o}, \mathbf{a}, \mathbf{o}')_i\}$

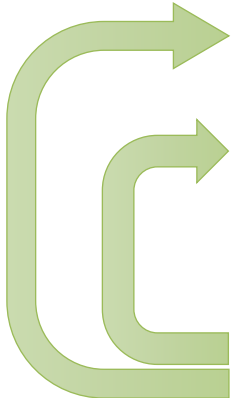
2. learn $p_\phi(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$, $p_\phi(r_t|\mathbf{s}_t)$, $p(\mathbf{o}_t|\mathbf{s}_t)$, $q_\psi(\mathbf{s}_t|\mathbf{o}_t)$

3. plan through the model to choose actions

4. execute the first planned action, observe resulting $\mathbf{o}'$ (MPC)

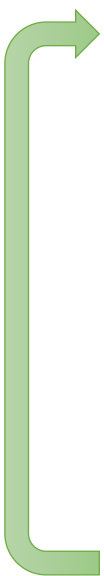
5. append $(\mathbf{o}, \mathbf{a}, \mathbf{o}')$ to dataset $\mathcal{D}$

every N steps



Actor-critic with learned representations

actor-critic with learned representations:

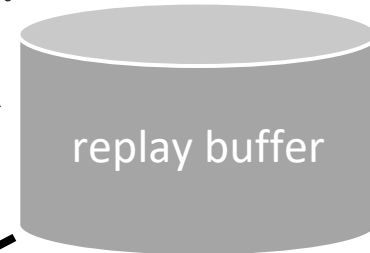
- 
1. get $(\mathbf{o}_i, \mathbf{a}_i, \mathbf{o}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{o})$
 2. update $p_\phi(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$, $p_\phi(r_t|\mathbf{s}_t)$, $p(\mathbf{o}_t|\mathbf{s}_t)$, $q_\psi(\mathbf{s}_t|\mathbf{o}_t)$
 3. infer $\mathbf{s}_i \sim q_\psi(\mathbf{s}_i|\mathbf{o}_i)$, $\mathbf{s}'_i \sim q_\psi(\mathbf{s}'_i|\mathbf{o}'_i)$
 4. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{o}'_i)}[\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]]$
 5. update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
 6. $\nabla_\theta J(\theta) \approx \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{o}_i)}[\nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{o}_i) \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})]$
 7. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

an *even better* idea with a full filtering/smoothing posterior

$$q_\psi(\mathbf{s}_t, \mathbf{s}_{t+1} | \mathbf{o}_{1:t+1}, \mathbf{a}_{1:t})$$

$$q_\psi(\mathbf{s}_t, \mathbf{s}_{t+1} | \mathbf{o}_{1:H}, \mathbf{a}_{1:H})$$

store $\mathbf{o}_i, \mathbf{a}_i, \mathbf{o}'_i$



use batch $\{(\mathbf{o}_i, \mathbf{a}_i, \mathbf{o}'_i)\}_{i=1}^B$ from buffer

why might we want to use this?

why might we prefer a full filtering/smoothing posterior?

Actor-critic + model-based RL

actor-critic with learned representations:

1. get $(\mathbf{o}_i, \mathbf{a}_i, \mathbf{o}'_i)$ by taking one step with $\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{o})$
2. update $p_\phi(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$, $p_\phi(r_t|\mathbf{s}_t)$, $p(\mathbf{o}_t|\mathbf{s}_t)$, $q_\psi(\mathbf{s}_t|\mathbf{o}_t)$
3. infer $\mathbf{s}_i \sim q_\psi(\mathbf{s}_i|\mathbf{o}_i)$, $\mathbf{s}'_i \sim q_\psi(\mathbf{s}'_i|\mathbf{o}'_i)$
4. simulate additional data using p_ϕ
5. evaluate $y_i = r(\mathbf{s}_i, \mathbf{a}_i) + \gamma \mathbb{E}_{\mathbf{a}' \sim \pi_\theta(\mathbf{a}'|\mathbf{o}'_i)} [\hat{Q}_\phi^\pi(\mathbf{s}'_i, \mathbf{a}')]]$
6. update ϕ using $\nabla_\phi \sum_{i=1}^B \|\hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a}_i) - y_i\|^2$
7. $\nabla_\theta J(\theta) \approx \sum_i \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\mathbf{a}|\mathbf{o}_i)} [\nabla_\theta \log \pi_\theta(\mathbf{a}|\mathbf{o}_i) \hat{Q}_\phi^\pi(\mathbf{s}_i, \mathbf{a})]$
8. $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$

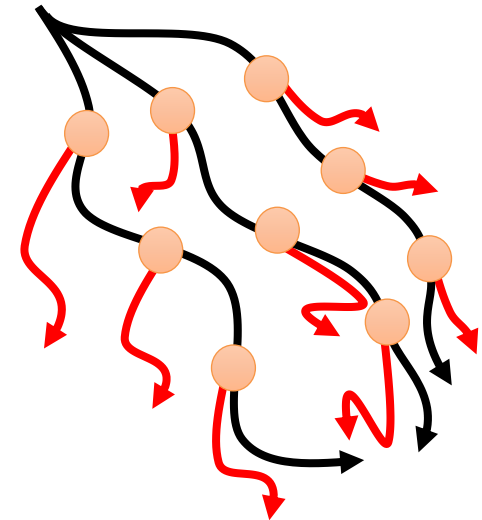
often prefer full latent space learning with $\pi_\theta(\mathbf{a}|\mathbf{s})$

such that $\pi_\theta(\mathbf{a}|\mathbf{o}) = E_{\mathbf{s} \sim q_\psi(\mathbf{s}|\mathbf{o})} [\pi_\theta(\mathbf{a}|\mathbf{s})]$

store $\mathbf{o}_i, \mathbf{a}_i, \mathbf{o}'_i$

replay buffer

use batch $\{(\mathbf{o}_i, \mathbf{a}_i, \mathbf{o}'_i)\}_{i=1}^B$ from buffer



Representation learning and model-based RL

Alex X. Lee^{1,2} Anusha Nagabandi¹ Pieter Abbeel¹ Sergey Levine¹
¹University of California, Berkeley
²DeepMind
{alexlee_gk,nagaban2,pabbeel,svlevine}@cs.berkeley.edu



Danijar Hafner *	Timothy Lillicrap	Jimmy Ba	Mohammad Norouzi
University of Toronto	DeepMind	University of Toronto	Google Brain
Google Brain			

